
boolean



CSS GRID

Il sistema definitivo per i layout bidimensionali

CSS Grid

Cos'è? Perché?

Prima di Grid, persino Flexbox (nato per layout monodimensionali) veniva spesso forzato per creare intere griglie di pagina.

Le specifiche di Grid sono state rilasciate e supportate massicciamente dai browser a partire dal 2017 fino a raggiungere un supporto nativo eccellente nei browser moderni.

Rappresenta il punto d'arrivo definitivo per la gestione dei layout complessi.



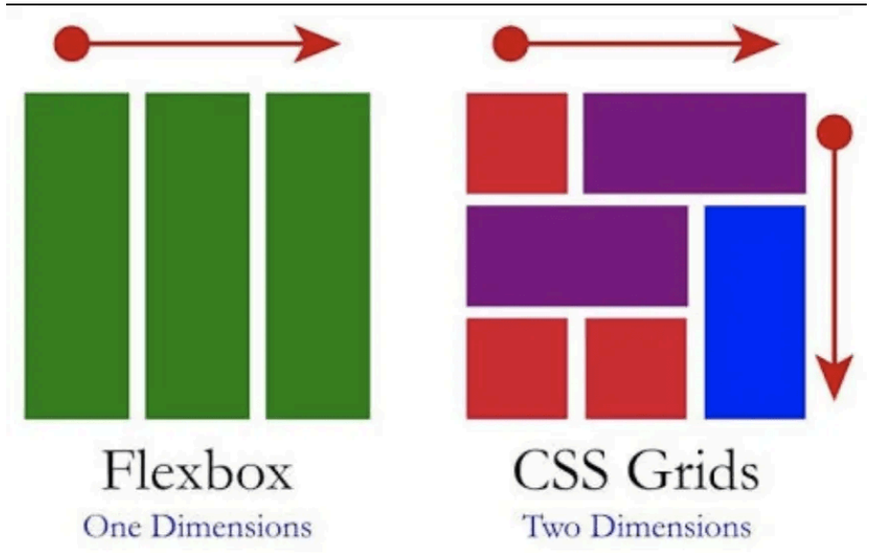
CSS Grid & Felxbox

Alleati, non nemici

Grid non sostituisce Flexbox, bensì lo affianca!

Grid: è progettato per la macro-struttura della pagina: il layout principale a 2 dimensioni.

Flexbox: è ideale per la disposizione e l'allineamento dei micro elementi: ad esempio l'allineamento interno di un menù di navigazione o il contenuto di una card.





Potremo utilizzare Flex in alcune parti della pagina, sfruttandolo solo dove serve e strutturare con Grid il layout principale della pagina stessa!

CSS Grid

Che cos'è Grid?

Il CSS3 **Grid**, è una modalità di layout bidimensionale composta da righe e colonne.

Consente di definire regole precise per posizionare gli elementi in aree specifiche dello schermo.

Introduce due concetti fondamentali:

Il **Grid-Container:** il tag genitore che definisce lo spazio e la struttura della griglia.

I **Grid-Item**: Gli elementi figli **diretti** posizionati al suo interno

CSS Grid

Display: grid è una proprietà del genitore!!!

La proprietà **display: grid** va data al tag genitore (**grid-container**).
Una volta creato il container, lo spazio viene automaticamente suddiviso per il numero dei suoi figli diretti, che diventeranno automaticamente **grid-item.**

```html
<div class="container-grid">
    <div class="item">1</div>
    <div class="item">2</div>
    <div class="item">3</div>
</div>
```

```css
.container-grid {
    display: grid;
}
```



⚠️ **Solo i figli diretti del Grid-Container subiscono l'effetto del posizionamento sulla griglia. I nipoti o i sub-elementi successivi rimangono nel normale flusso del documento.**

```html
<div class="container-grid"> <!-- genitore Flex -->

    <div class="red"> <!-- Flex-item -->
       <div class="figlio-di-red">
       </div>
    </div>

    <div class="blue"></div> <!-- Flex-item -->

    <div class="green"></div> <!-- Flex-item -->

</div>
```

🤔 *Nota avanzata:* Per estendere la griglia ai livelli successivi, i CSS moderni mettono a disposizione la proprietà specifica `display: subgrid`

CSS Grid

La rivoluzione Bidimensionale



- Controllo totale e simultaneo sia della direzione orizzontale che verticale, nessun bisogno di scegliere l'asse.
- Possibilità di sovrapporre gli elementi intenzionalmente all'interno delle celle senza ricorrere a position: absolute.
- Totale indipendenza tra l'ordine degli elementi nel codice HTML e la loro posizione visiva finale sullo schermo.

CSS Grid

L'Anatomia della griglia: un po' di terminologia

La griglia è composta da 4 elementi principali:

Grid Lines: le linee divisorie (orizzontali e verticali) che creano la griglia numerata.

Grid Track: Lo spazio tra due linee adiacenti (ovvero una singola riga o una singola colonna).

Grid Cell: L'unità minima della griglia, data dall'intersezione tra una riga e una colonna.

Grid Area: Una porzione rettangolare della griglia formata da una o più celle unite.

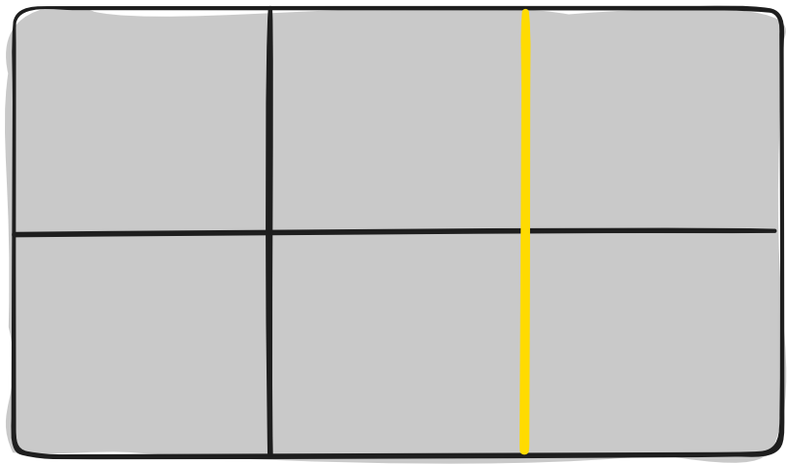
CSS Grid

Grid Line

Le righe che compongono la struttura della griglia.

Possono essere verticali oppure orizzontali e si trovano ai lati di una riga o di una colonna.

Qui la linea gialla è un esempio di grid line di una colonna.



CSS Grid

Grid Track

Lo spazio compreso tra due grid lines adiacenti.

Puoi pensarle come le colonne o le righe della griglia.

Nell’esempio, viene mostrata la grid track tra la seconda e la terza grid line orizzontali



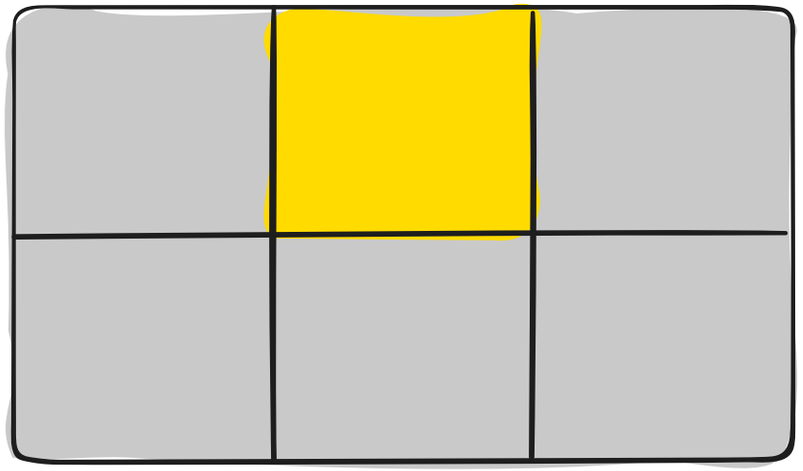
CSS Grid

Grid Cell

Lo spazio compreso tra due grid line orizzontali adiacenti e due grid line verticali adiacenti.

È la singola “unità” della griglia.

Nell’esempio, è la cella compresa fra le grid line delle righe 1 e 2 e delle colonne 2 e 3.



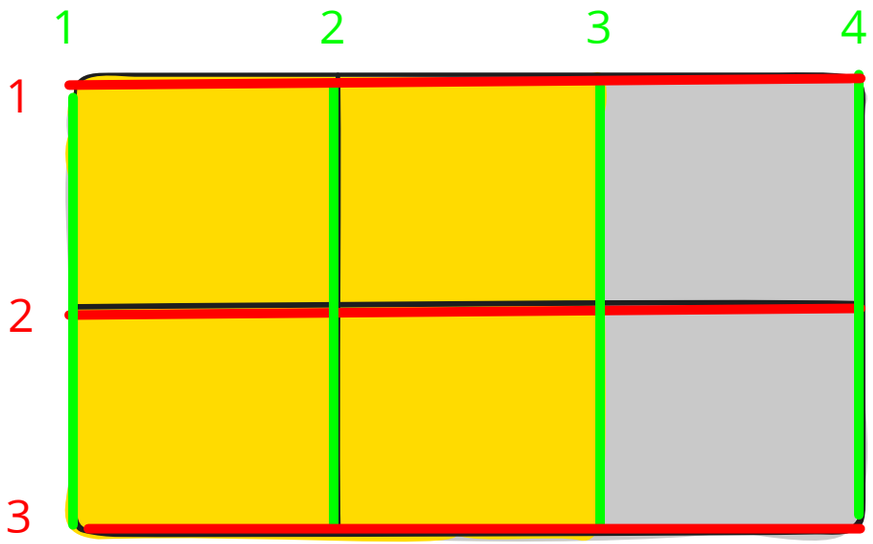
CSS Grid

Grid Area

Lo spazio totale delimitato da quattro linee di griglia.

Una Grid area può essere composta da una o più celle di griglia.

Nell'esempio, è l'area gialla compresa fra le grid line 1 e 3 orizzontali e 1 e 3 verticali.





Definire la griglia

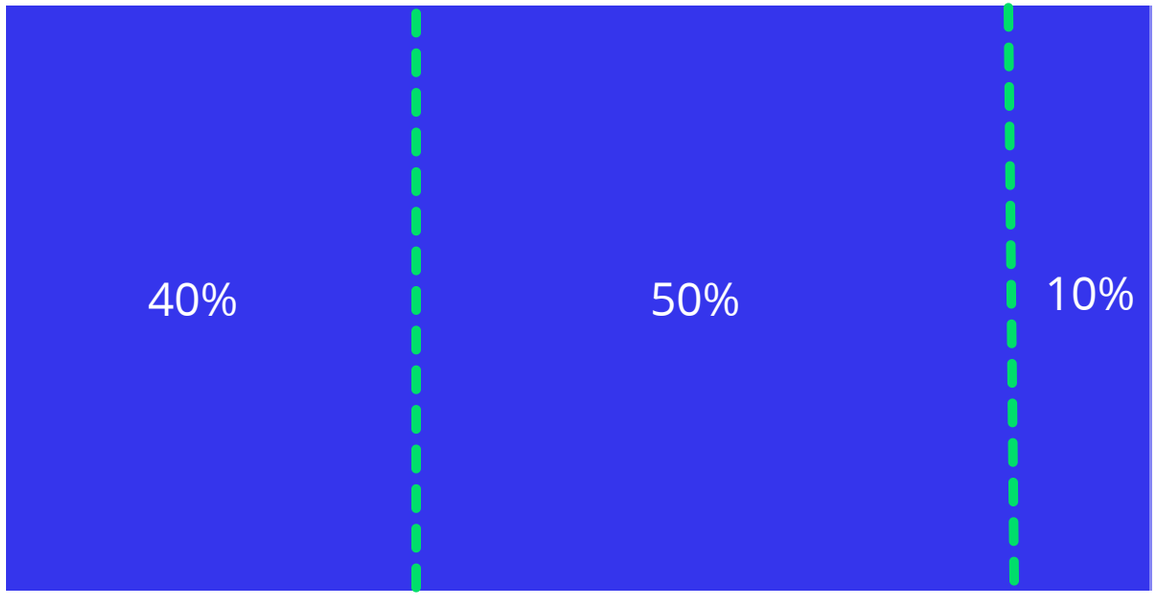
Definiamo la griglia

grid-template-columns- Grid-container rule

Definisce il numero e la larghezza delle colonne all'interno della griglia.

Accetta diverse tipologie di valori: px, %, fr

```
1 .container {
2   display: grid;
3   grid-template-columns: 40% 50% 10%;
4 }
```



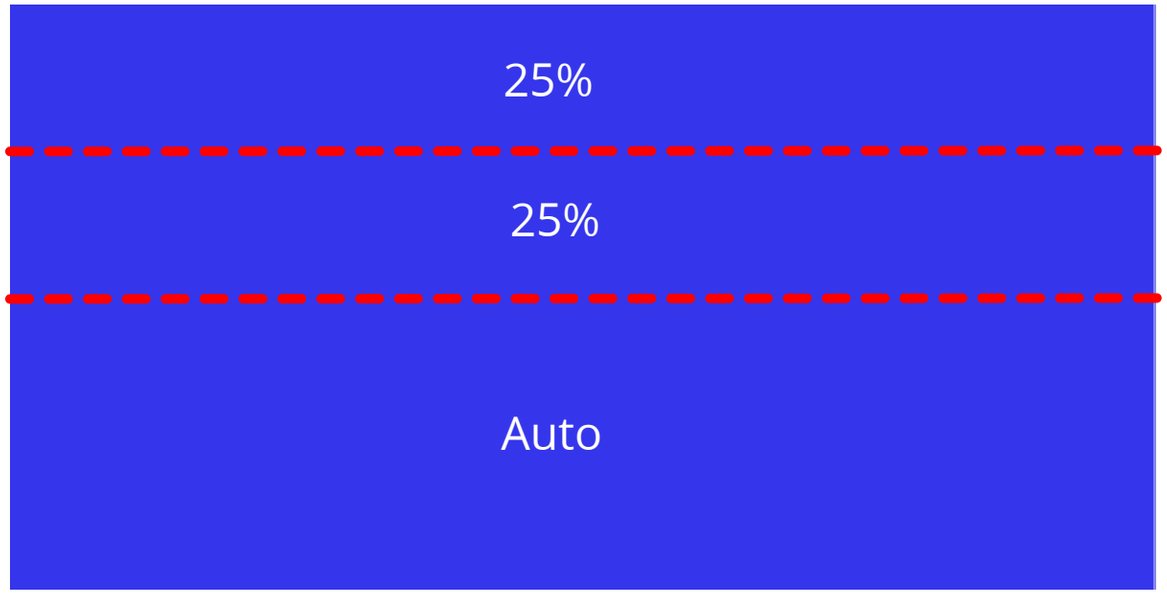
Definiamo la griglia

grid-template-rows- Grid-container rule

Definisce il numero e la larghezza delle righe all'interno della griglia.

Anche questa può accettare diversi valori.

```
1 .container {
2   display: grid;
3   grid-template-rows: 25% 25% auto;
4 }
```



Definiamo la griglia

Proprietà Gap

È una nuova unità di misura introdotta per Grid ed ora estesa anche a Flexbox.

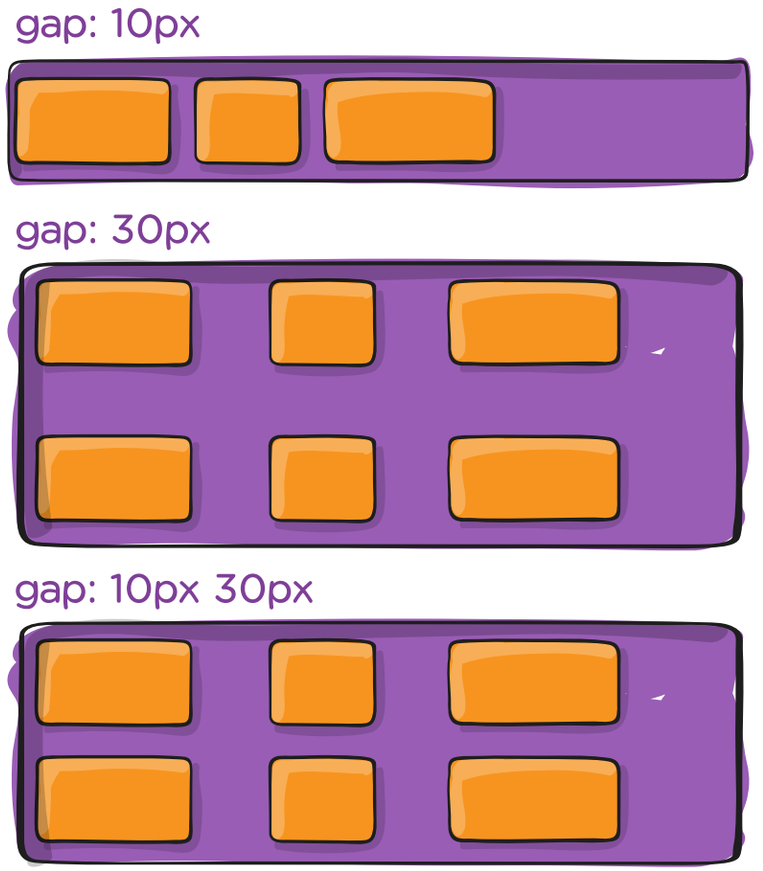
Permette di impostare lo spazio vuoto (i canali o "gutters") tra le righe e le colonne, senza ricorrere ai margini.

Può essere declinato in modo specifico:

`row-gap`: 15px; (spazio tra le righe)

`column-gap`: 20px; (spazio tra le colonne)

Oppure sintetizzato nella proprietà scorciatoia **`gap`**: 15px 20px; (o un solo valore se uguale).



Definiamo la griglia

L'Unità Frazionaria (fr)

È una nuova unità di misura introdotta appositamente per CSS Grid.

Rappresenta una frazione dello spazio disponibile rimasto libero nel grid container.

```css
.container {
  display: grid;
  grid-template-columns: 1fr 2fr 1fr;
}
```



Definiamo la griglia

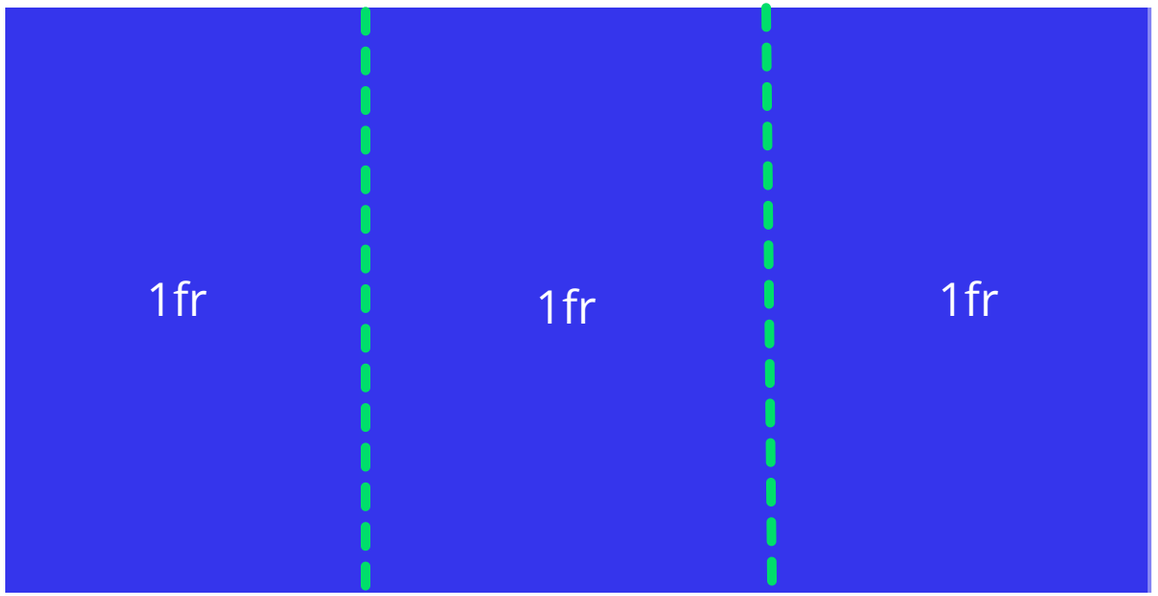
La funzione repeat() (fr)

È un nuovo metodo di CSS Grid. per evitare di ripetere codice inutilmente.

Rappresenta una frazione dello spazio disponibile rimasto libero nel grid container.

```css
.container {
  display: grid;
  grid-template-columns: 1fr 1fr 1fr;
}
```

```css
.container {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
}
```





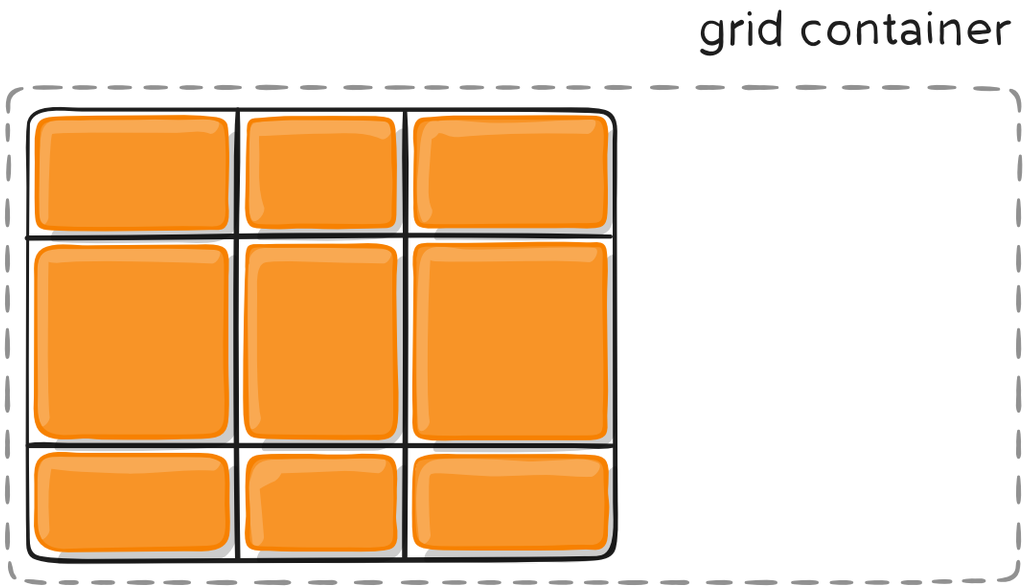
LIVE CODING

Spostiamo la griglia

justify-content - Grid-container rule

Si applica al contenitore quando l'intera griglia ha dimensioni totali inferiori rispetto al container per spostare o distribuire l'intera griglia sull'asse orizzontale.

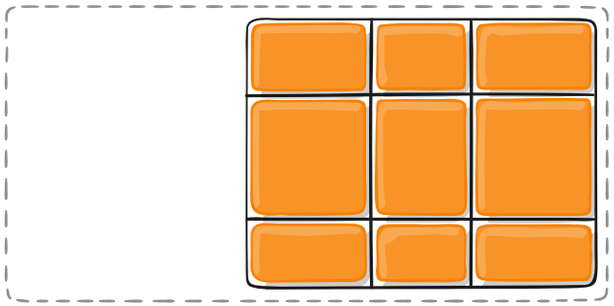
```
1 .container {
2  justify-content: start;
3 }
```



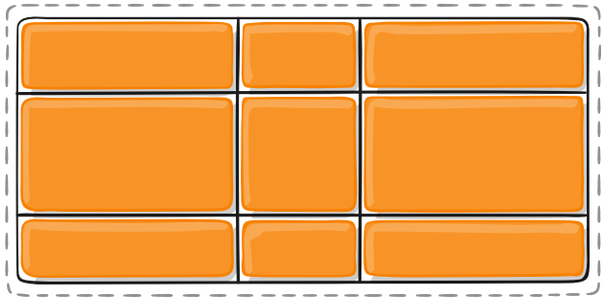
Spostiamo la griglia

justify-content - Grid-container rule

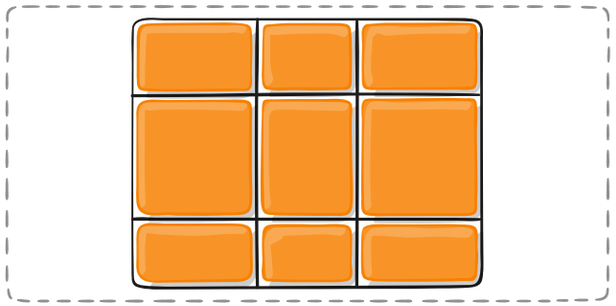
`justify-content: **end**;`



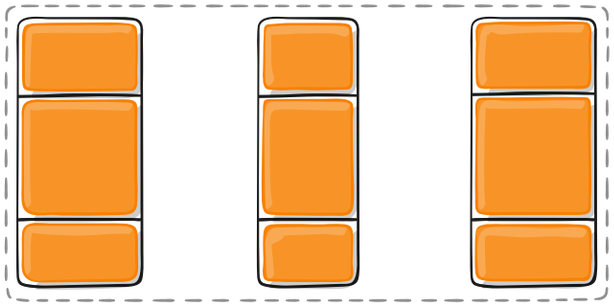
`justify-content: **stretch**;`



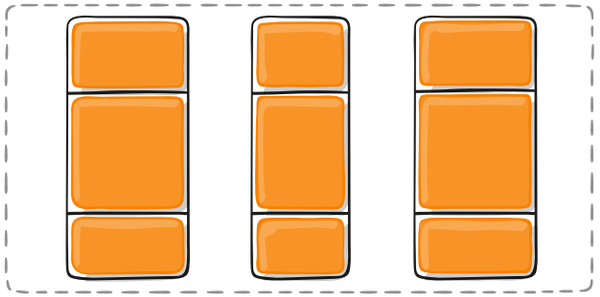
`justify-content: **center**;`



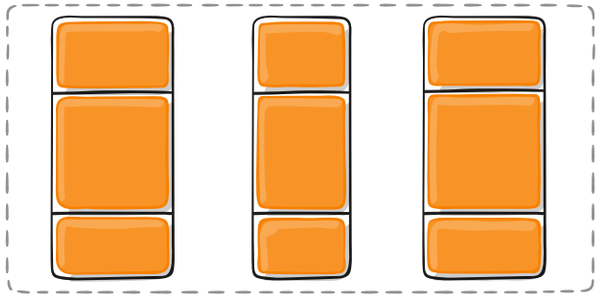
`justify-content: **space-between**;`



`justify-content: **space-evenly**;`



`justify-content: **space-around**;`

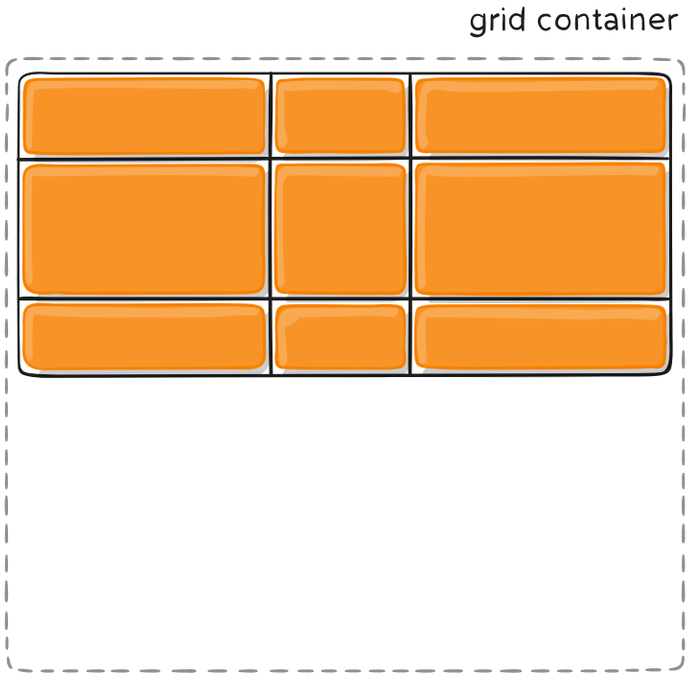


Spostiamo la griglia

align-content - Grid-container rule

Si applica al contenitore quando l'intera griglia ha dimensioni totali inferiori rispetto al container, per spostare o distribuire l'intera griglia sull'asse verticale.

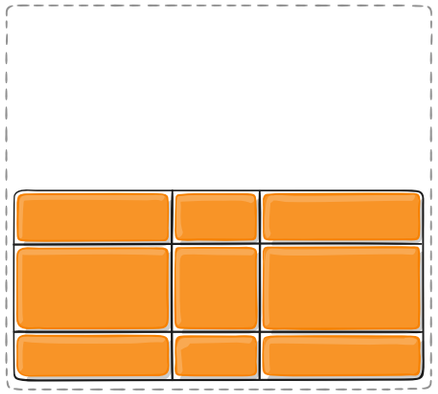
```
1 .container {
2  align-content: start;
3 }
```



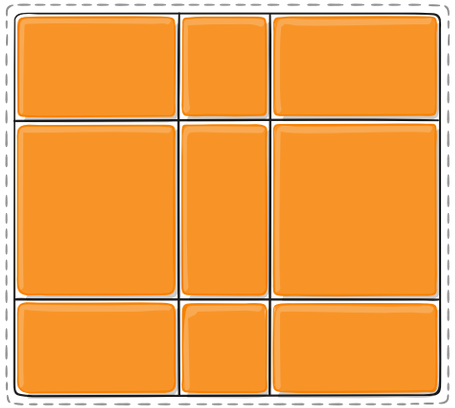
Spostiamo la griglia

align-content - Grid-container rule

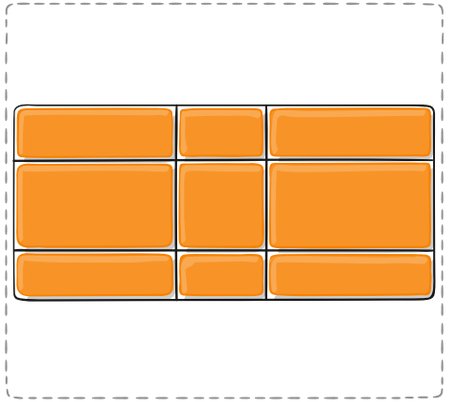
`align-content: **end**;`



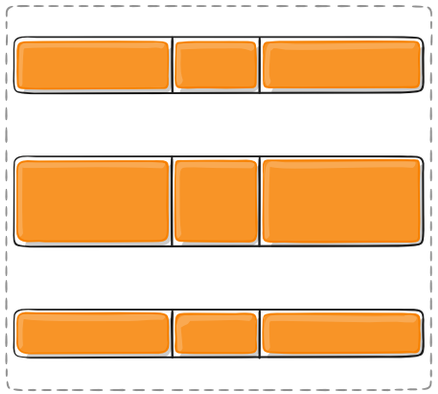
`align-content: **stretch**;`



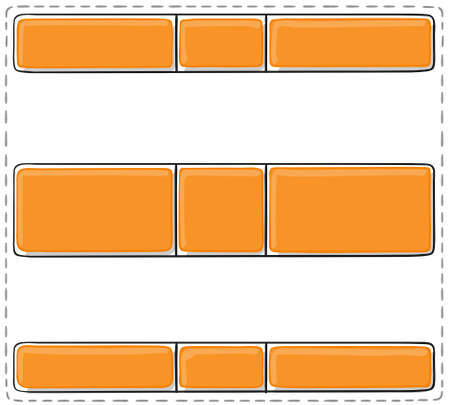
`align-content: **center**;`



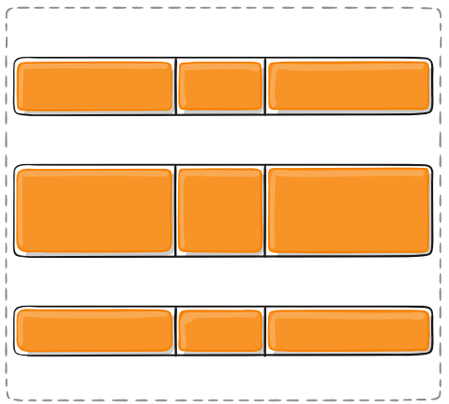
`align-content: **space-around**;`



`align-content: **space-between**;`



`align-content: **space-evenly**;`



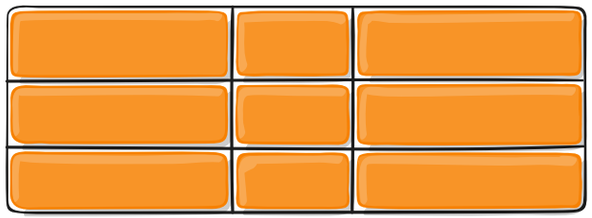
Gestiamo la posizione dei contenuti

Allineiamo i Grid Item

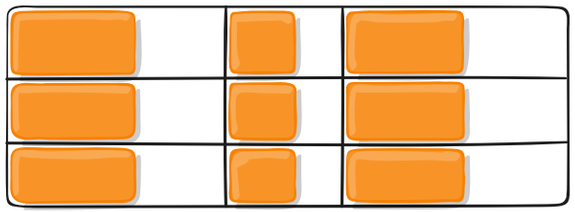
proprietà justify-items Grid-container rule

Definisce il comportamento di allineamento degli items all'interno delle loro rispettive celle, secondo l'asse orizzontale.

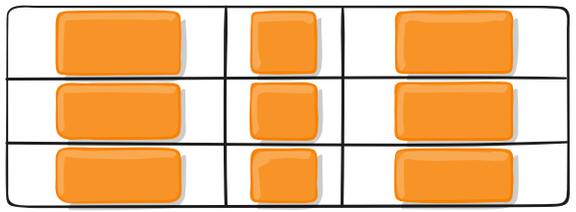
`justify-items: **stretch;** (default)`



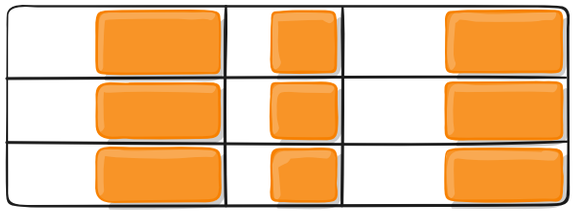
`justify-items: **start;**`



`justify-items: **center;**`



`justify-items: **end;**`

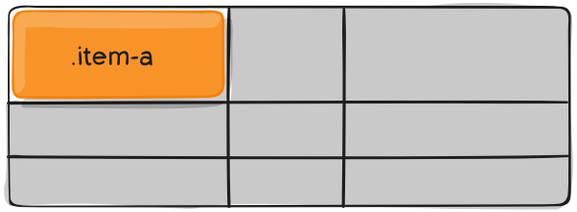


Allineiamo i Grid Item

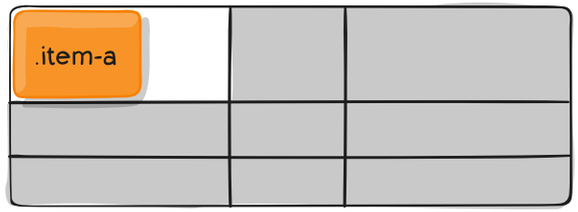
proprietà justify-self Item-container rule

Se vogliamo che un singolo item si disponga nella sua cella diversamente rispetto agli altri, possiamo usare su di lui **`justify-self`** e dare un allineamento diverso.

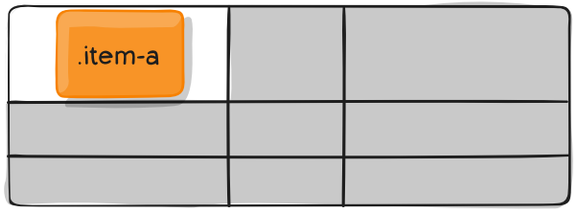
`justify-self: stretch;`



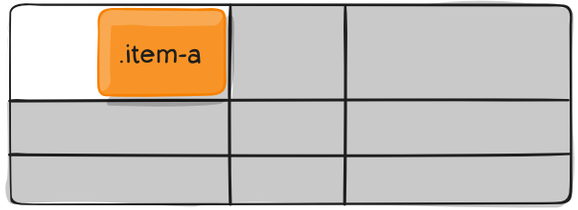
`justify-self: start;`



`justify-self: center;`



`justify-self: end;`

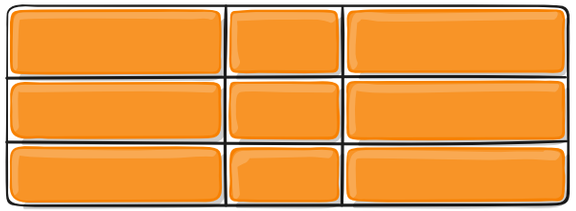


Allineiamo i Grid Item

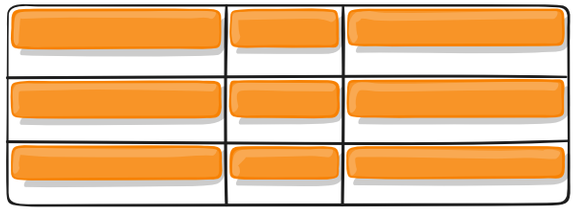
proprietà align-items Grid-container rule

Definisce il comportamento di allineamento degli items all'interno delle loro rispettive celle, secondo l'asse verticale.

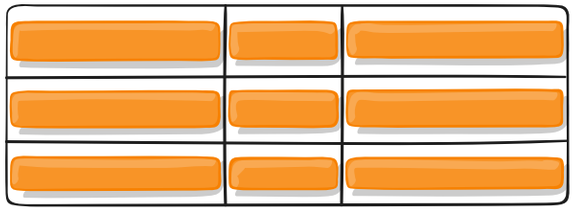
`align-items: **stretch;** (default)`



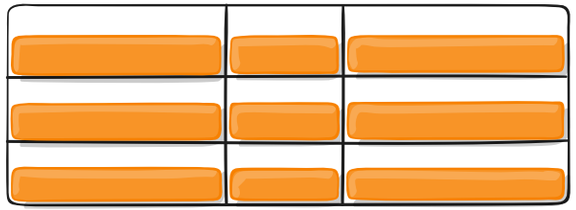
`align-items: **start;**`



`align-items: **center;**`



`align-items: **end;**`

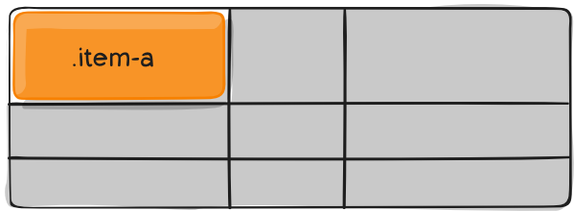


Allineiamo i Grid Item

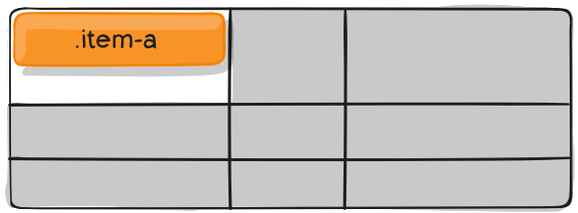
proprietà align-self Item-container rule

Se vogliamo che un singolo item si disponga nella sua cella diversamente rispetto agli altri, possiamo usare su di lui **`align-self`** e dare un allineamento diverso.

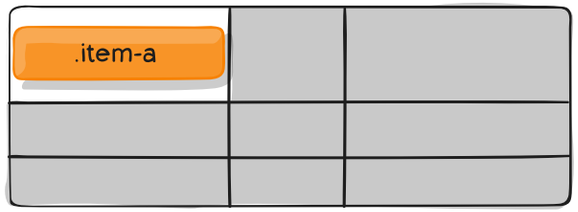
`align-self: stretch;`



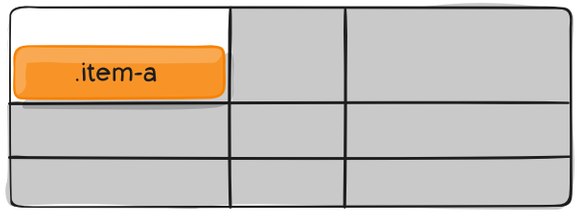
`align-self: start;`



`align-self: center;`



`align-self: end;`



Spostare i Grid Item: `grid-column`

Proprietà grid-column Grid-item rule

Proprietà da applicare sui singoli **tag figli** (grid-item) per definirne l'estensione orizzontale.Utilizza i numeri delle *Grid Lines* per indicare dove un elemento deve iniziare e finire.

```
1 .mio-item {
2   grid-column: 1 / 3;
3   /* Parte dalla linea 1 orizzontale
4    e finisce alla linea 3 orizzontale
5   */
6 }
```

Si può usare anche la sintassi **`span`** per indicare quante celle occupare

(es. **`grid-column: 1 / span 2`**).

Spostare i Grid Item: `grid-column`

Proprietà grid-row Grid-item rule

Proprietà da applicare sui singoli **tag figli** (grid-item) per definirne l'estensione verticale.

Utilizza i numeri delle *Grid Lines* per indicare dove un elemento deve iniziare e finire.

```
1 .mio-item {
2   grid-row: 1 / 2;
3   /* Parte dalla linea verticale 1
4     e finisce alla linea verticale 2 */
5 }
```

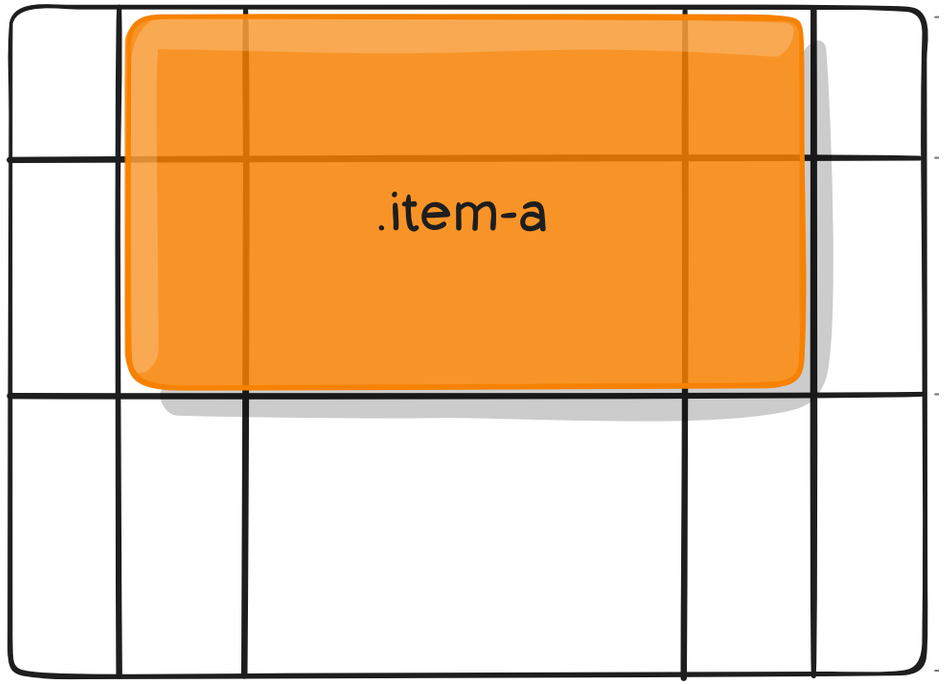
Anche qui possiamo utilizzare **`span`**

Spostare i Grid Item: `grid-column`

```css
.grid-container{
  display:grid;
  grid-template-rows: 25% 100px  auto
  grid-template-columns: 40px 50px auto 50px 40px;
}


.item-a {
  grid-row-start: 1;
  grid-row-end: 3;
  grid-column-start: 2;
  grid-column-end: 5;
}

/* Shortand */
.item-a {
  grid-row: 1 / 3;
  grid-column: 2 / 5;
}
```





LIVE CODING

Consigli per allenarsi!

1. Cheat sheet completo:

https://css-tricks.com/complete-guide-css-grid-layout/

2. Grid Garden:

https://cssgridgarden.com/#it

3. Grid Attack:

https://codingfantasy.com/games/css-grid-attack

4. W3Schools

https://www.w3schools.com/css/css_grid.asp